

Catálogo Interativo

Pokédex Interativa

Tema escolhido: **Pokémon** — PokéAPI

Dado	Valor
Curso	Bacharelado em Sistemas de Informação (BSI)
Disciplina	Desenvolvimento Mobile Híbrido
Professor responsável	Mark Joselli
Turma	[PREENCHER: número/código da turma]
Data de entrega	20/09/2026
Nome do projeto	Pokédex Interativa
Tema escolhido	Pokémon (PokéAPI)

Integrantes do grupo

#	Nome completo
1	Rogério de Abreu Mar
2	Logan Borges
3	Felipe da Veiga Gomes
4	Alex Narok Stavas

Confira se cada nome acima está **exatamente igual** ao que a pessoa diz no início do vídeo. Pelo item 8 do enunciado, integrante cujo nome completo não constar nesta capa recebe **nota zero**, mesmo aparecendo na gravação.

Seção 1 — Links

Item	Link
Repositório GitHub (público)	https://github.com/0DevAbreu/CatalogoInterativoFlutter
Vídeo (YouTube, público)	[PREENCHER: colar a URL do vídeo no YouTube]

O repositório foi verificado e está público. Antes de enviar, abra os dois links em uma aba anônima para confirmar que abrem sem login.

Seção 2 — Tema e Configuração

Item	Configuração adotada
Tema	Pokémon. O verbo “consumido” do enunciado foi adaptado para capturado : a lista de itens consumidos é a Pokédex pessoal do usuário.
API utilizada	PokéAPI — https://pokeapi.co/api/v2 . Pública, gratuita e sem chave de acesso. Endpoints usados: <code>/pokemon?limit=&offset=</code> (listagem paginada), <code>/pokemon/{id}</code> (detalhe) e <code>/pokemon-species/{id}</code> (descrição da Pokédex).
Persistência	Local + Nuvem (bônus). Baseline com <code>shared_preferences</code> ; sincronização em nuvem com Supabase (Postgres), tabela <code>user_items</code> com Row Level Security por usuário. Estratégia local-first: a marcação é gravada no aparelho e só depois enviada à nuvem, então o app funciona offline.
Login	Autenticação real (bônus) com Supabase Auth (e-mail e senha), com a coleção vinculada ao usuário autenticado. Sem as credenciais compiladas, o app cai automaticamente no login local (baseline), com a senha guardada como hash SHA-256.
Segurança	As credenciais do Supabase não estão no repositório: são injetadas em tempo de compilação via <code>--dart-define-from-file=env.json</code> , e o <code>env.json</code> está no <code>.gitignore</code> (há um <code>env.example.json</code> versionado como modelo).

Seção 3 — Autoavaliação dos Requisitos Funcionais

RF	Implementado	Caminho do arquivo principal
RF01 — Catálogo em GridView com “Carregar mais”	Sim	lib/screens/home_screen.dart lib/providers/catalog_provider.dart
RF02 — Navegação para a tela de detalhes	Sim	lib/widgets/pokemon_card.dart lib/screens/detail_screen.dart
RF03 — Tela de detalhes (2ª requisição à API)	Sim	lib/screens/detail_screen.dart lib/services/pokeapi_service.dart
RF04 — Favoritos com Provider	Sim	lib/providers/collection_provider.dart
RF05 — Tela de favoritos	Sim	lib/screens/favorites_screen.dart
RF06 — Persistência local + nuvem (bônus)	Sim	lib/services/local_storage_service.dart lib/services/sync_service.dart supabase/schema.sql
RF07 — Login e itens capturados (autenticação real, bônus)	Sim	lib/screens/login_screen.dart lib/screens/captured_screen.dart lib/services/auth_service.dart
RF08 — Busca com navegação direta ao detalhe	Sim	lib/screens/home_screen.dart (método _search)
RF09 — Feedback de UI (progresso, FutureBuilder, erros)	Sim	lib/widgets/state_views.dart lib/screens/detail_screen.dart
RF10 — Acessibilidade	Sim	Transversal: lib/widgets/pokemon_card.dart lib/widgets/pokemon_grid.dart lib/core/theme/app_theme.dart

Evidências. O app foi executado em emulador Android (Pixel 7, API 36) e o fluxo completo foi percorrido: cadastro e login via Supabase Auth, catálogo, paginação até o #040, detalhes, favoritar, capturar, busca válida e inválida, fechamento e reabertura do app (persistência) e queda de rede durante o “Carregar mais”. O projeto também possui **24 testes automatizados** (flutter test) e passa no flutter analyze sem advertências.

Seção 4 — Tema da Explicação Técnica

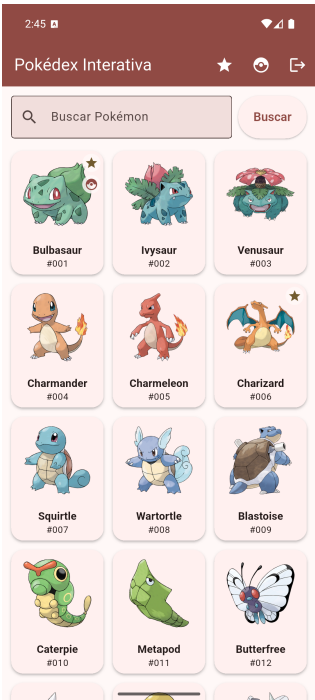
Item	Resposta
Tema escolhido	Provider vs. setState — por que favoritos e itens capturados são gerenciados como estado global, e não como estado local de widget.
Minuto do vídeo	A explicação técnica começa em 5:30 e vai até 8:00 .
Resumo do argumento	A mesma marcação de favorito aparece em quatro pontos da interface (selo no card, estrela na tela de detalhes, tela de Favoritos e contagem no título). Com setState, cada tela teria a própria cópia e seria preciso propagar o resultado pelo Navigator.pop. Centralizamos em um ChangeNotifier (CollectionProvider), usando context.watch nas telas de lista e context.select no card, para reconstruir apenas o item alterado. Estado que vive e morre em uma única tela (texto da busca, future do detalhe) permaneceu em setState.

Seção 5 — Screenshots das telas principais

Capturas feitas no emulador Android (Pixel 7, API 36) com o app conectado à PokéAPI e ao Supabase.



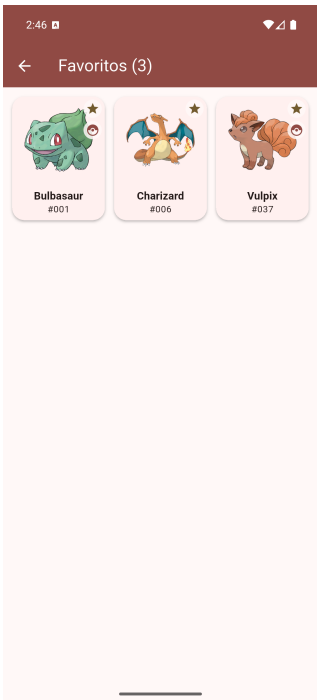
1. Login



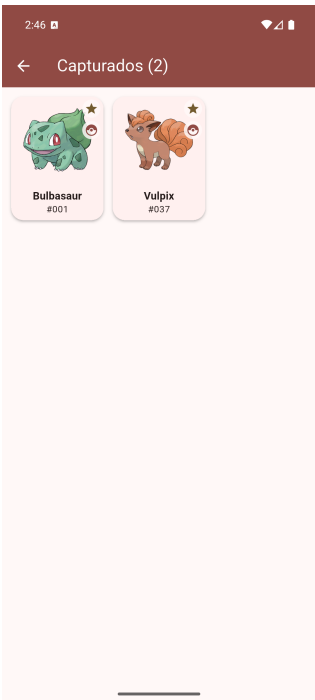
2. Catálogo (RF01)



3. Detalhes (RF03)



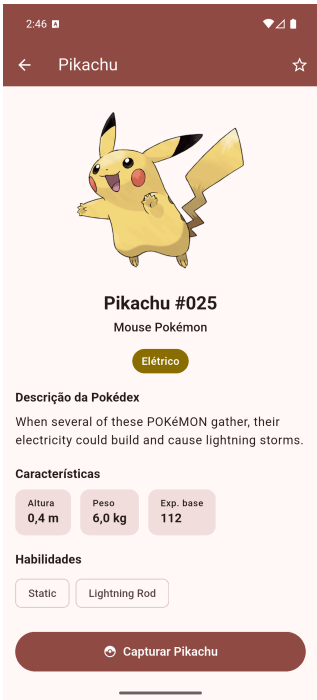
4. Favoritos (RF05)



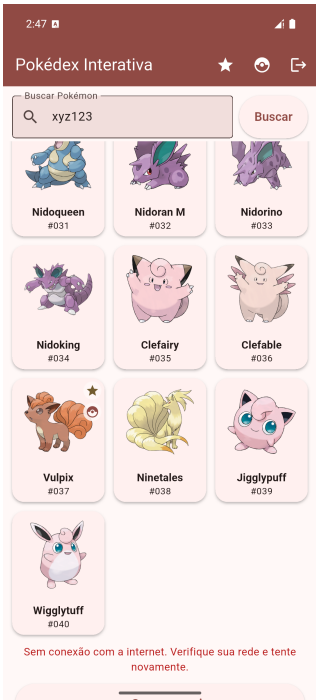
5. Capturados (RF07)

Anexo — Evidências complementares

Não exigidas pelo item 5, mas incluídas como evidência dos requisitos de busca, tratamento de erro e acessibilidade.



Busca por “pikachu” (RF08)



Falha de rede no “Carregar mais” (RF09)



Fonte do sistema em 200% (RF10)